

Originally published on [the Retool blog](#) on 11 August 2026.

I lead product for Governance and Infrastructure at Retool, and most technology leaders I talk to have already won the argument that governance should enable AI rather than block it. That's not the hard part anymore. Here, AI governance means enforcing clear limits at runtime, the moment an AI tool reaches real data and takes a real action.

The numbers say almost no one is there yet. In our [State of AI Governance 2026 survey](#) of 307 CTOs, CIOs, and CISOs, just 8% rated their internal AI governance as "strong," and only 5% were confident they could see what was actually running in production. At the same time, 55% said they want governance controls in one platform rather than reconfigured app by app. The gap between "we have a policy" and "it's enforced in the flow of work" is where most of the risk sits.

[!A slide titled "The visibility gap is the governance gap" showing statistics: 8% call their internal AI governance strong, 5% are confident they see what AI is running in production, and 55% want AI controls in one platform.](#)

You don't close that gap with a longer policy document. You close it by being able to answer three questions about any AI workflow, and by enforcing the answers in the system rather than in a binder.

The three questions:

1. Who's acting? 2. What can they reach? 3. Can you stay in control?

That's the whole framework. For each one there's a question worth asking and a trap worth avoiding.

[Watch the talk](#)

1. Who's acting on your data? The three identities

For decades there was one kind of actor asking for data: a human, scoped to a role. AI introduced two more, and the mistake is treating all three the same.

AI Identity Whose permission it uses Primary risk How to scope it	--- --- --- ---	Human
Their own role Role creep Least privilege, scoped to the use case	Agent on a human's permission	
A person's access, borrowed Scales at machine speed; weak attribution Same least-privilege scope as the person, plus its own audit trail	Agent with its own service identity Its own account, no human	
Over-permissioned "god-mode" access Deliberately narrow scope, expanded on purpose, never broad by default		

[!"Three kinds of identity touch your data," listing Human, Agent as the user, and Agent its own service user, with the latter highlighted as the highest-risk identity.](#)

The practical point is simple. You can't scope, audit, or trust an action until you know which of these took it. Most teams fail at one of two extremes. They over-permission, handing people and agents god-mode access. Or they get nervous about that and under-permission so hard that people route around the system and build in the shadows. Both are governance failures. The fix is the boring discipline that predates AI by decades: least privilege, scoped to the use case, expanded deliberately. It applies to a service agent the same way it applies to an employee.

2. What can AI agents access? Environment, platform, and application

"Reach" is bigger than data. It's also about actions. Can this actor submit the refund, write to the record, post something customer-facing? Access gets dangerous the moment it can turn into action, because permission sprawl becomes action sprawl. This is the reach that governance actually has to control, not just the data, but what an actor can do with it.

It helps to think about access as three nested boundaries, and to notice that you define access in one place but enforce it in another:

- **Environment.** Where the data and the AI actually run, and who's isolated from whom. The strongest boundary is your own environment, self-hosted or in your own cloud, where you set who and what can reach what and segregate teams logically inside it.
- **Platform.** Where you define access once: role-based access on the resources, apps, workflows, and agents, plus the data policies for which rows, columns, and records each can touch. This is your governance control plane, where policy is authored.
- **Application (runtime).** Where those rules are enforced, on every request, in every app: the row a given user sees, the column that's masked, the records filtered out by region or sensitivity. This is the data plane, the point where governance is enforced, not authored.

This is the part most teams miss. You define access once at the platform layer, and it's enforced at runtime inside every application. You never re-implement governance per app. Do that and your governance scales. Skip it and it breaks the first time someone spins up an app you didn't know about.

[!Diagram illustrating three nested layers of access control: Environment for data separation, Platform control plane for granular role-based access control \(RBAC\), and Application runtime for enforcing rules.](#)

Two principles make this layer of governance hold.

Write the rule once; enforce it everywhere. This is where the current wave of AI-assisted building tends to break governance. You can spin up an impressive app in any of the new builders, but the access rules written into that app live only in that app. At a hundred apps and agents, that doesn't scale and it's impossible to audit. The rule has to live a layer down, on the serving side, so it applies to every app, automation, and agent that touches the data. The question to ask any platform is whether

governance is specific to one use case, or whether you can set it once and have it enforced across everything.

Make permissions granular, not broad. Role-based access only works if you can break a role into specific scopes. Coarse roles force the same over-permission-or-shut-it-down choice from the first question. We've seen it consistently. When large teams can scope access precisely to the use case, they ship more with their tools and agents, not less, because the secure path stops being the slow path.

[!User interface for configuring data access policies, demonstrating how rules are written once to block tables, mask columns, and enforce row-level security across all requests.](#)

3. How do you stay in control of AI agents at runtime?

Staying in control of an AI workflow comes down to three things you enforce at runtime: rules a model can't override, an audit trail for every action, and a hard cap on spend. You've already established who's acting and what they can reach. This last question is whether you'd know, and could step in, if something went sideways at runtime.

Keep the rules deterministic

This is the one I'd underline. Don't trust a model to enforce its own permissions. A stochastic system will, eventually and apologetically, do the thing you told it not to: "oops, I didn't realize I couldn't." Use an LLM to help author policy from your compliance rules if you want. Just don't let an agent decide at runtime what it's allowed to do. The hard rules are deterministic and the platform enforces them.

Watch the agent with another agent

One emerging pattern we like is a "guardian" agent that knows the deterministic rules and checks another agent's work: scanning for unmasked PII, flagging out-of-policy actions, reporting back. It's early, but the direction is clear. The same capability that created the risk becomes part of the control.

Trust, but verify

Audit trails are the backbone here. The goal isn't to block everything. It's to let work happen while keeping every action attributable: who or what did it, what it touched, and whether it was approved. Run analysis on those logs to spot access patterns. Don't just store them.

Two dimensions get left out of "control" and shouldn't.

- **Data stays put.** Sensitive data shouldn't leave your perimeter while agents act on it. Per-user and per-agent identity, masking applied at runtime, and self-hosting keep it in your environment. That also supports the data-residency and sovereignty posture that regulations like GDPR increasingly push toward.

- **Spend stays bounded.** An agent that can act can also spend. We've seen teams burn through an annual AI budget in a single quarter. Cost is a governance dimension now. Budget by use case, not just by person. Route to fit-for-purpose models instead of running the most expensive model for every trivial task. And tie spend to an outcome so you can defend the ROI.

[!Slide about "Governing the agent at runtime," detailing principles: deterministic rules over LLM permissions, guardian agents for compliance, and a "trust but verify" audit approach.](#)

What you can do to improve governance today

You don't need to fix everything at once. Pick one real, high-value workflow and run the three questions against it:

- **Who's acting** on it: human, agent-as-user, or agent-as-service? And where do its permissions come from?
- **What can it reach**, and is that access layered and scoped, or broad by default?
- **Can you stay in control**: deterministic rules, audit trails, and a cap on spend?

If the answers come easily, you're governing AI well. If they don't, you've just found your roadmap. And when you evaluate a platform to build on, hold it to the same three questions. The right answer is one place to govern wall to wall, one that can also absorb whatever your teams build elsewhere, instead of a stitched-together set of one-off controls.

AI isn't going to wait for governance to catch up. The organizations that win won't be the ones that governed the least, and they won't be the ones that locked everything down. They'll be the ones that made trust fast, by answering these three questions where the work actually happens.

[!The takeaway: Who's acting, what they can reach, and can you stay in control — answer all three where the work happens.](#)

Get the full data in Retool's [State of AI Governance 2026](#).